



PROPUESTA TÉCNICA ARQUITECTÓNICA

CapitalFlow — Modernización de Plataforma Financiera B2B

Campo	Valor
Preparado para	Carlos Mendoza, CTO — CapitalFlow
Elaborado por	Francisco Liébanas
Cargo	Angular Expert Candidate, Nter
Fecha	Mayo 2026
Versión	9.0
Clasificación	CONFIDENCIAL

CONFIDENCIAL — Para uso interno y presentación al Board e Inversores

ÍNDICE

SECCIÓN 1 · Executive Summary
SECCIÓN 2 · Diagnóstico de Situación Actual
SECCIÓN 3 · Arquitectura Objetivo
SECCIÓN 4 · Plan de Optimización de Rendimiento
SECCIÓN 5 · Estrategia de Seguridad
SECCIÓN 6 · Roadmap de Refactoring Incremental
SECCIÓN 7 · Librería de Componentes Compartidos
SECCIÓN 8 · Modernización del Despliegue
SECCIÓN 9 · Plan de Ejecución (6 meses)
SECCIÓN 10 · Análisis de Riesgos
SECCIÓN 11 · Métricas de Éxito
ANEXO A · Respuesta detallada a la pregunta del inversor
ANEXO B · Mapa de archivos del prototipo

Cierre

Pulsa cualquier entrada del índice para saltar a esa sección.

SECCIÓN 1 · Executive Summary

CapitalFlow es una plataforma SaaS de gestión financiera B2B que opera en 12 países, sirve a **850 clientes corporativos** y factura **€18M anuales**. La compañía negocia una ronda Serie B de **€40M** cuyo due-diligence técnico se celebra en 8 semanas.

Tras seis años de crecimiento sin inversión en arquitectura, la plataforma presenta tres riesgos estructurales que comprometen la ronda: **fragilidad arquitectónica** (8 equipos, versiones distintas de Angular, un equipo en React, pipeline monolítico de 14 minutos con 28 % de fallos), **rendimiento inaceptable** (bundle 2,8 MB, FCP 9,2 s, tablas que congelan el navegador 18 s) y **11 vulnerabilidades de seguridad alta o crítica** que bloquean la certificación SOC 2 Type II.

Esta propuesta presenta un plan de **6 meses, €450 000, equipo de 1 consultor + 2 arquitectos senior + 14 desarrolladores**, ejecutable sin detener el negocio. La modernización se basa en cuatro pilares: arquitectura **Micro Frontends con Module Federation**, integración del equipo React mediante **Custom Elements API** sin forzar migración, **librería de componentes compartidos** que elimina las siete implementaciones de DataGrid actuales, y **CI/CD automatizado con zero-downtime** para clientes regulados.

El plan está **validado mediante prototipo funcional** que demuestra Angular 17 + Angular 18 + React 18 conviviendo en el mismo runtime, tabla de 120 000 filas con virtual scroll, exportación Excel en hilo secundario sin bloqueo de interfaz, 7 security headers nginx, pipeline GitLab con 18 jobs y librería compartida de 19 componentes con Storybook.

Compromisos medibles ante el Board y los inversores

Hito	Plazo	Compromiso verificable
Due-diligence Serie B	Semana 8	Prototipo funcional, métricas baseline → target y plan documentado
Certificación SOC 2 Type II	≤ Semana 6	11 vulnerabilidades remediadas o documentadas con plan
Mejora de rendimiento	Semana 8	FCP < 4 s, bundle < 800 kB, tabla 80 K < 1 s
Pipeline CI/CD operativo	Semana 4	Tasa de éxito > 95 %, deploys independientes por equipo
Plataforma productiva final	Semana 26	FCP < 2,5 s, bundle < 300 kB, tests > 50 %, los 8 equipos con pipeline propio

SECCIÓN 2 · Diagnóstico de Situación Actual

Problemas priorizados por urgencia e impacto

#	Problema	Impacto en negocio	Urgencia
1	Pipeline monolítico de 14 min	Un deploy de Pagos bloquea 7 equipos. Incidente de 6 h en Compliance, notificación formal de incumplimiento de SLA a cliente regulado en Alemania.	CRÍTICA
2	11 vulnerabilidades alta o crítica	Certificación SOC 2 Type II bloqueada. Inversores con copia del informe preliminar de la auditoría externa.	CRÍTICA
3	FCP 9,2 s · bundle 2,8 MB	Cliente enterprise en Francia (€420 K/año) amenaza con no renovar contrato si no hay mejora demostrable.	ALTA
4	Tabla 80 K registros congela 18 s	Módulo de conciliaciones bancarias inutilizable para clientes corporativos grandes.	ALTA
5	8 equipos: Angular 16/17/18 + React 18	Coordinación imposible. Tiempo medio de deploy 4-5 h. Reescritura de Analytics inviable (3 años de desarrollo).	ALTA
6	108 componentes > 800 LOC, 12 servicios "dios"	Una feature de 2 días lleva 6-8. Change Detection: 180+ ciclos por interacción.	MEDIA
7	7 DataGrid distintos, 5 Modal	Inconsistencias visuales en cada feedback de cliente. Sin design tokens unificados pese a tener Figma.	MEDIA
8	Deploy manual 4-5 h, 28 % de fallos	Sin garantía de continuidad para clientes regulados. Rollback nocturno de 2-3 h.	ALTA

Causas raíz

- **Falta de aislamiento entre módulos:** el repo monolítico hace que cualquier deploy bloquee al resto de equipos.
- **Cero estandarización:** cada equipo escogió librería UI, patrón de estado y proceso de release distinto.
- **Pipeline manual sin tests automatizados:** explica el 28 % de fallos y los rollbacks nocturnos.
- **Cero design system unificado:** cada equipo reinventó componentes básicos con bugs propios.
- **Renderizado directo al DOM sin sanitización:** causa de las vulnerabilidades XSS detectadas.
- **Cero security headers ni CSP:** la plataforma no respeta la mínima defensa moderna en navegador.
- **Cookies sin flags de seguridad:** requiere coordinación con backend, ya planificada para Sprint 1.

Métricas baseline contra targets

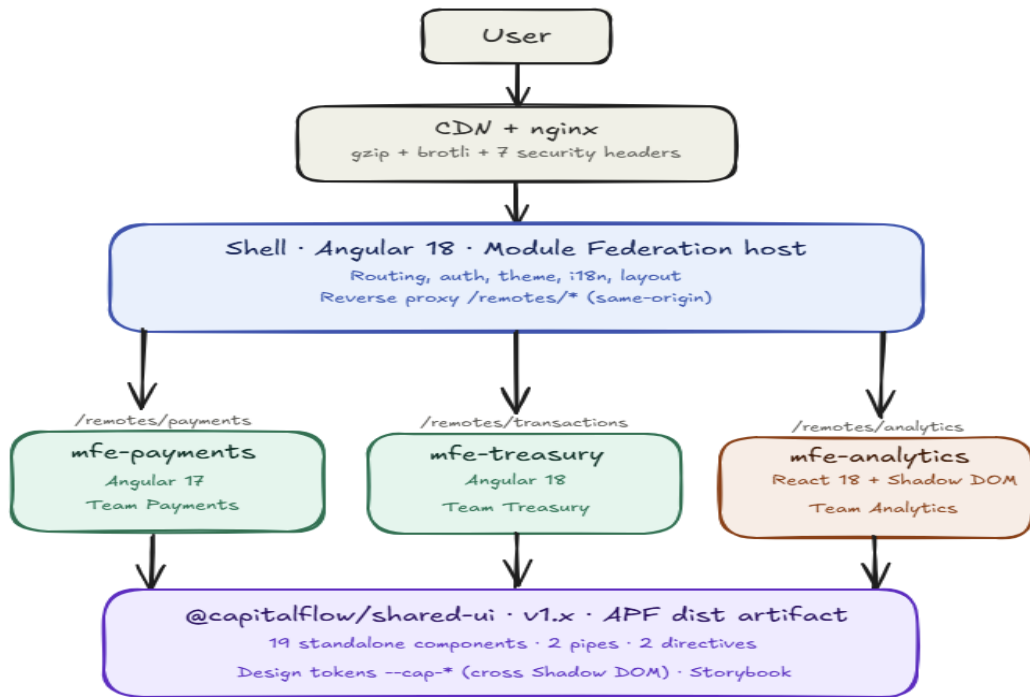
Métrica	Baseline	Target Sem 8	Target Sem 26
Bundle inicial	2,8 MB	< 800 kB	< 300 kB
First Contentful Paint en 4G	9,2 s	< 4 s	< 2,5 s
Time to Interactive en WiFi corporativo	14 s	< 6 s	< 3 s
Tabla conciliaciones 80 K filas	18 s congela	< 1 s	< 300 ms
Exportación Excel 120 K filas	Bloquea UI	Worker sin bloqueo	< 5 s sin bloqueo
Cobertura de tests	3 %	> 30 %	> 50 %
Tasa de fallo de deploy	28 %	< 10 %	< 1 %
Tiempo de deploy productivo	4-5 h	30 min	< 5 min/MFE
Vulnerabilidades alta/crítica	11	≤ 2	0
Ciclos Change Detection / interacción	180+	< 40	< 20

SECCIÓN 3 · Arquitectura Objetivo

La arquitectura objetivo se basa en **Micro Frontends con Module Federation** (Webpack 5). Cada módulo de negocio es una aplicación independiente que se compila, despliega y versiona por su propio equipo. El **Shell** (host) actúa como punto único de navegación, autenticación, theming e idioma. Los Micro Frontends se cargan dinámicamente en runtime mediante `loadRemoteModule()`.

Diagrama de la arquitectura objetivo

El siguiente diagrama representa el flujo desde el navegador del usuario hasta cada Micro Frontend, pasando por el reverse proxy del Shell que mantiene la same-origin policy y comparte design tokens vía CSS Custom Properties:



Cómo leer el diagrama: el Shell actúa como reverse proxy y carga cada MFE bajo `/remotes/{nombre}/` mediante Module Federation (`loadRemoteModule()`). Esto preserva la same-origin policy (sin CORS, cookies cross-domain ni problemas con Web Workers). Cada MFE se compila a un Custom Element (`<mfe-payments>`, `<mfe-treasury>`, `<mfe-analytics>`) que Angular consume como HTML nativo con `CUSTOM_ELEMENTS_SCHEMA`. El MFE de Analytics añade Shadow DOM para aislar sus estilos React; los CSS Custom Properties del Shell (`--cap-primary`, `--cap-bg...`) cruzan la barrera por herencia, garantizando consistencia visual cross-framework. La librería `@capitalflow/shared-ui` se distribuye a los 3 consumidores Angular como artefacto dist (formato APF), eliminando duplicación de DataGrid, Modal y notificaciones.

Componentes de la arquitectura

Capa	Componente	Responsabilidad	Estado
Edge	CDN + nginx (gzip + brotli)	Cacheo estático, compresión, security headers	Implementado
Host	Shell Angular 18	Routing, autenticación, theme claro/oscuro, idioma ES/EN, layout, reverse proxy /remotes/*	Implementado
MFE	mfe-payments (Angular 17)	Pagos internacionales — equipo Payments	Implementado
MFE	mfe-treasury (Angular 18)	Gestión de tesorería y conciliaciones bancarias — equipo Treasury — CDK Virtual Scroll 120 K filas, Web Worker XLSX	Implementado
MFE	mfe-analytics (React 18)	Dashboards e informes — equipo Analytics — Custom Element con Shadow DOM	Implementado

Capa	Componente	Responsabilidad	Estado
MFE	mfe-compliance (Angular 16→18)	Reporting regulatorio — equipo Compliance	Plan Sprint 1-2
MFE	mfe-onboarding (Angular 17)	Alta de clientes — equipo Onboarding	Plan F5
MFE	mfe-admin (Angular 17)	Panel de administración — equipo Admin	Plan F5
MFE	mfe-auth (Angular 18)	Autenticación y permisos — equipo Auth	Plan F5
Librería	@capitalflow/shared-ui v1.x	19 componentes compartidos, design tokens, Storybook	Implementado

Nota sobre el alcance del prototipo: el prototipo entregado implementa **3 MFEs como prueba arquitectónica** (mfe-payments en Angular 17, mfe-treasury en Angular 18 y mfe-analytics en React 18), suficientes para demostrar la coexistencia de versiones de Angular distintas y la integración React vía Custom Elements. mfe-treasury implementa precisamente el peor caso del briefing: la tabla de conciliaciones bancarias de 80 K-120 K filas. Los **4 MFEs restantes** se construyen progresivamente: **Compliance migra a Angular 18 en Sprint 1-2** (prioritario por SOC 2 y por estar en versión sin LTS), y **Onboarding, Admin y Auth** entran en F5 replicando el patrón ya validado.

Decisiones de arquitectura y alternativas descartadas

Decisión	Justificación	Alternativa descartada y motivo
Module Federation (Webpack 5)	Carga dinámica en runtime, dependencias compartidas, deploys independientes por equipo.	iframes: aislamiento excesivo, sin estado compartido, UX fragmentada.
Shell como host único	Punto central de navegación, autenticación, design tokens y theming.	Multi-repo sin shell: fragmentación de UX y sin SSO unificado.
Custom Elements API para React	Estándar W3C, cero cambios al código React de Analytics, Shadow DOM aísla CSS.	Reescribir Analytics en Angular: 3 años de trabajo perdido y rechazo del equipo.
Reverse proxy /remotes/* en nginx	Carga MFEs same-origin → evita CORS, cookies cross-domain y problemas con Web Workers.	CORS abierto: complica autenticación y headers de seguridad.
Standalone Components (Angular 15+)	Menos boilerplate, mejor tree-shaking, compatible con Angular 16/17/18.	NgModules: patrón obsoleto, mayor acoplamiento.
Signals + OnPush universal	Reactividad nativa sin RxJS para estado simple, change-detection óptimo.	BehaviorSubject + markForCheck manual: más código y memory leaks.
shared-ui como artefacto dist	Consumer importa solo desde el entry point principal; ng-packagr emite Angular Package Format.	Path mapping al source: obligaba a tener node_modules de la librería en cada consumer.

Decisión	Justificación	Alternativa descartada y motivo
CSS Custom Properties como design tokens	Theming cross-framework sin acoplamiento, atraviesa Shadow DOM con fallbacks.	CSS-in-JS: incompatible entre React y Angular.

Coexistencia Angular 16/17/18 + React 18

El patrón Module Federation + Custom Elements es **agnóstico a la versión Angular**. El prototipo demuestra Angular 17 + Angular 18 + React 18 conviviendo. El equipo **Compliance está en Angular 16, versión sin LTS desde noviembre 2024**: su migración a Angular 18 es prioritaria por SOC 2 y se ejecuta en Sprint 1-2 mediante `ng update @angular/core@17 @angular/cli@17` y luego `ng update @angular/core@18` (saltos automatizados por los migrators oficiales de Angular). Los demás equipos restantes se integran sin riesgo arquitectónico.

La integración del equipo React es la decisión más relevante. **El detalle técnico completo de la respuesta a la pregunta del inversor sobre el equipo Analytics se desarrolla en el Anexo A**, con el patrón Custom Element + Shadow DOM + sincronización de tema e idioma.

Autenticación y SSO cross-MFE

Una decisión crítica con 8 equipos en frameworks distintos es **cómo se comparte la sesión** entre Shell y MFEs. La estrategia evita la fragmentación actual:

1. **Token JWT centralizado**: el Shell lo persiste en `httpOnly` cookie y lo expone en runtime mediante `AuthBridgeService`. El equipo Auth (F5) gestiona emisión y refresh.
2. **Inyección al MFE por atributos del Custom Element**: `<mfe-treasury auth-token="..." user-context="...">`. Los MFEs leen en `attributeChangedCallback` (estándar W3C, igual en Angular y React).
3. **Eventos custom para refresh y logout**: `capitalflow:auth-refresh` y `capitalflow:auth-expired` sobre document. Sin event bus propietario, sin librería compartida. Compatible SOC 2.

Implementación: el `AuthBridgeService` está stubbed con mock JWT en el prototipo. Integración real con backend en Sprint 1 junto con la cookie policy de la Sección 5.

Beneficios para el negocio

- **Autonomía por equipo**: cada MFE se despliega independientemente. El equipo de Pagos puede publicar un fix urgente sin coordinar con los otros 7 equipos.
- **Cero migración forzada**: el equipo Analytics sigue trabajando en React. Los demás equipos migran en su propio Sprint cuando les toca.
- **Time-to-market reducido**: el ciclo de release pasa de "una vez cada cuatro semanas el monolito" a "cuando cada equipo lo necesita".
- **Resiliencia**: la caída de un MFE no afecta al resto de la plataforma. El Shell muestra mensaje degradado controlado.
- **Reaprovechamiento de Figma**: el design system del equipo de diseño se materializa en `@capitalflow/shared-ui`.

SECCIÓN 4 · Plan de Optimización de Rendimiento

La estrategia de rendimiento se ataca en cuatro frentes: **bundle size**, **tiempos de carga**, **procesamiento de grandes volúmenes de datos** y **soporte a navegadores legacy**. Las cinco técnicas siguientes están **implementadas y verificadas** en el prototipo.

Estrategias y resultados verificados

Estrategia	Mecanismo	Resultado en el prototipo
Lazy loading por ruta	Cada módulo de negocio se carga bajo demanda con <code>loadComponent()</code> . El Shell inicial solo descarga navegación, autenticación y theming.	Bundle inicial del Shell: ~549 kB raw / ~140 kB tras gzip (vs 2,8 MB del cliente). Cada wrapper de MFE pesa < 2 kB.
CDK Virtual Scroll	Solo se renderizan ~18 filas visibles en el DOM independientemente del volumen total del dataset.	Tabla de 120 000 registros sin congelar. Métrica <code>domNodeCount</code> expuesta en runtime confirma 18 filas activas.
Web Worker para exportación XLSX	La generación del Excel ocurre en un hilo dedicado mediante <code>new Worker()</code> . El hilo principal sigue respondiendo a la interacción del usuario.	Exportación de 120 000 filas sin bloquear UI. El usuario puede seguir navegando durante el proceso.
Signals + OnPush universal	Todos los componentes usan <code>ChangeDetectionStrategy.OnPush</code> . Estado reactivo con <code>signal()</code> , <code>computed()</code> y <code>effect()</code> de Angular 18.	Ciclos de Change Detection reducidos drásticamente vs el patrón <code>BehaviorSubject</code> + <code>markForCheck</code> manual.
Reverse proxy same-origin	El Shell expone los MFEs bajo <code>/remotes/{nombre}/</code> vía nginx. En desarrollo, <code>webpack-dev-server</code> lee <code>proxy.conf.json</code> .	Evita CORS. Permite que los Web Workers vivan en el mismo origen que la página y compartan cookies.

Soporte a navegadores legacy (IE11, 6 % de usuarios bancarios)

El briefing especifica un 6 % de usuarios en IE11 (clientes regulados que no pueden actualizar navegadores corporativos). La modernización completa de IE11 sobre Angular 18 es **inviabile técnicamente** (Angular 18 requiere navegadores evergreen).

La solución es una **estrategia de doble portal**:

1. **Modal bloqueante para navegadores legacy**: el Shell detecta `documentMode` (IE) antes de cargar Angular y muestra un overlay modal a pantalla completa que impide interactuar con la aplicación. Implementado en el prototipo en `shell/public/cap-modal-legacy.{js,css}` como HTML+CSS+JS estático (única forma técnica: Angular 18 no se ejecuta en IE11, por lo que un componente Angular nunca llegaría a renderizarse). La redirección efectiva al portal legacy se entrega como parte del bundle conjunto modal+portal en Sprint 5; el prototipo cumple ya con el bloqueo total, no con la redirección a una URL que no existe aún.
2. **Portal legacy paralelo**: una versión simplificada (solo lectura: ver saldos, descargar informes) en HTML estático, mantenida como activo congelado hasta que estos clientes migren navegador. Coste estimado: 2 desarrolladores × 4 semanas en Sprint 5.
3. **Comunicación proactiva al cliente**: notificación del soporte limitado de IE11 con plan de migración de su lado, documentado en el contrato de renovación.

Métricas y herramientas de medición

KPI	Herramienta	Cadencia
Bundle size	webpack-bundle-analyzer + budgets en <code>angular.json</code>	Cada PR

KPI	Herramienta	Cadencia
FCP, TTI, LCP, CLS	Lighthouse CI	Semanal en main
Tabla 80 K filas	Chrome DevTools Performance + métrica domNodeCount	Cada release
Exportación XLSX	Custom metric exportPhase + Profiler	Cada release
Ciclos Change Detection	Angular DevTools Profiler	Mensual
Cobertura de tests	Karma coverage report (Istanbul)	Cada PR (gate del CI)

Service Worker para clientes en redes limitadas

El briefing señala que el **35 % de los usuarios trabaja en filiales de mercados emergentes con conectividad limitada**. La estrategia es introducir un **Service Worker con `@angular/pwa`** (Sprint 4) con cuatro estrategias diferenciadas:

1. **`prefetch` para assets estáticos** (JS, CSS, fuentes, traducciones): chunks lazy-loaded en background tras primer login. TTI de visitas posteriores < 1 s.
2. **`freshness` con `maxAge` para datos transaccionales**: red preferente, fallback a caché en < 1 s con indicador visual **"datos no actualizados"**.
3. **`performance` para CDN** (logos, design tokens, imágenes): caché primero, actualización en background.
4. **Exclusión de APIs financieras + update flow controlado**: `/api/transactions/*`, `/api/payments/*` excluidos del caché (cumple SOC 2). Banner **"Hay versión nueva — recarga"** en lugar de update silencioso.

Beneficio cuantificable: para usuarios en mercados emergentes, FCP de visitas posteriores baja de ~9 s (red móvil) a < 1 s (caché local).

SECCIÓN 5 · Estrategia de Seguridad

La seguridad se aborda en tres planos: **remediación inmediata** de las 11 vulnerabilidades del informe externo, **estrategia preventiva** para que no vuelvan a aparecer, y **plan de formación del equipo** alineado con OWASP. La solución debe ser **compatible con el editor WYSIWYG** que usan los administradores.

Remediación de las 11 vulnerabilidades

#	Vulnerabilidad	Mitigación	Estado
1	XSS en comentarios de transacciones	Bindings <code>{{ }}</code> y <code>[innerText]</code> (Angular escapa automáticamente). Nunca <code>[innerHTML]</code> salvo <code>SafeHtmlPipe</code> con <code>DOMPurify</code> . Demo interactiva en <code>/admin/comments</code> con 4 comentarios seed (el 4º con <code><script></code> literal) + botón "Inyectar payload".	Implementado · Demo
2	iframe sin validación (javascript:)	Validación del protocolo (sólo <code>https:</code>) + allowlist de hosts confiables.	Implementado

#	Vulnerabilidad	Mitigación	Estado
3	Filenames de carga ejecutables	Interpolación con <code>{{ }}</code> (Angular sanitiza). Validación adicional de extensión y MIME type en cliente y backend.	Implementado
4	Reflected XSS en búsqueda global	Uso de <code>[innerText]</code> en lugar de <code>[innerHTML]</code> para reflejar términos. Inyección no se ejecuta.	Implementado
5	WYSIWYG acepta <code><script></code>	Sanitización con <code>DOMPurify.sanitize()</code> al guardar. Allowlist estricta: permite formato visual, bloquea scripts.	Implementado
6	Sin Content Security Policy	CSP estricta en nginx: <code>default-src self ; script-src self ; frame-ancestors none</code> . Hardening adicional (nonces, base-uri) en Sprint 1.	Implementado
7	Cookies sin HttpOnly ni Secure	Coordinación con backend para <code>Set-Cookie: HttpOnly; Secure; SameSite=Strict</code> .	Sprint 1 backend
8	Sin HSTS	<code>Strict-Transport-Security: max-age=31536000; includeSubDomains</code> en nginx.	Implementado
9	Sin X-Frame-Options, X-Content-Type-Options, Referrer-Policy	Las tres declaradas en <code>nginx.conf</code> .	Implementado
10	Sin Permissions-Policy	<code>camera=(), microphone=(), geolocation=()</code> en nginx.	Implementado
11	Sin auditoría externa SAST/DAST	<code>npm audit</code> , <code>gitleaks</code> y <code>Trivy</code> integrados en pipeline en Sprint 2 (junto con SOC 2 Type II).	Planificado Sprint 2

Estrategia preventiva (que no vuelvan a aparecer)

- **`SafeHTMLPipe` centralizado** en `@capitalflow/shared-ui` para todo HTML dinámico. Ningún equipo puede usar `[innerHTML]` directamente: la regla ESLint custom lo prohíbe excepto a través del pipe.
- **Regla ESLint custom** que detecta uso de `bypassSecurityTrust*` sin un comentario `// SECURITY: motivo adyacente`. PR con uso no justificado se bloquea automáticamente.
- **DOMPurify con allowlist estricta** — permite formato visual (`<b>`, `<i>`, `<a href>`), bloquea `<script>`, `data: URIs` y todo `on*= event handler`.
- **Pipeline CI/CD con tres gates de seguridad**: `npm audit --audit-level=moderate` (Sprint 1), `gitleaks` para credenciales en commits (Sprint 1) y `Trivy/Semgrep` para SAST (Sprint 2). Build se bloquea si encuentra crítico.
- **Auditoría externa anual** contractual con la firma actual. Renovación de SOC 2 Type II cada 12 meses.

SOC 2 y dependencias sin LTS: Angular 16 del equipo Compliance

El equipo Compliance opera con **Angular 16**, versión cuyo soporte LTS oficial terminó en noviembre 2024. Un auditor SOC 2 Type II marcaría esta dependencia como observación crítica si llegara a la auditoría sin un control compensatorio documentado. La estrategia es doble:

5. **Migración prioritaria a Angular 18 en Sprint 1-2** (semanas 1-4 del plan), antes de la auditoría SOC 2 prevista en semana 6-8. Se ejecuta con los migrators oficiales de Angular CLI (ng update @angular/core@17 @angular/cli@17 seguido de ng update @angular/core@18), pair-programming con un arquitecto senior asignado al equipo durante las 2 semanas de migración. Coste estimado: 2 desarrolladores senior × 2 semanas + arquitecto a media jornada = ~€18 000 dentro del presupuesto.
6. **Compensating control documentado mientras la migración está en curso:** SOC 2 acepta dependencias EOL si existe un plan de migración firmado, con fechas concretas, responsables identificados y métricas de progreso (commits, PRs, tests pasando en la nueva versión). Este documento se mantiene en el registro de riesgos del Anexo C y se actualiza semanalmente.

Por qué Compliance va primero y no último: el equipo Compliance es precisamente el que pone en riesgo SOC 2, por lo que su migración es prioritaria. El resto de equipos (Onboarding, Admin, Auth) están en Angular 17 o 18 con LTS vigente, su migración es operativa pero no urgente. Este orden — el outlier técnico primero — es lo que permite que la certificación SOC 2 se obtenga en el plazo de la due-diligence (8 semanas) sin compensating controls residuales en la auditoría final.

Plan de formación del equipo

La formación es **condición necesaria** para que la estrategia preventiva funcione. Sin formación, los desarrolladores siguen los patrones que conocen y vuelven a introducir las mismas vulnerabilidades.

Programa	Audiencia	Cadencia	Presupuesto
Workshop OWASP Top 10 para Angular y React	14 desarrolladores + 2 arquitectos	Inicial (semana 2) + refresco trimestral	€8 000 (formador externo + materiales)
Code review obligatorio con foco en seguridad	Todo el equipo	Cada PR; 2 revisores en archivos críticos (auth, sanitización)	Coste interno (incluido en plan de proyecto)
Curso de Secure Coding (Angular/React)	2 arquitectos senior + 1 desarrollador líder por equipo	Anual	€12 000 (3 certificaciones × €4 000)
Sesión mensual de "ataques recientes"	Todo el equipo, 30 min	Mensual: revisión de CVEs nuevos relevantes a Angular/React	Coste interno (rotación entre arquitectos)
Onboarding security para nuevos miembros	Nuevas incorporaciones	Antes del primer PR	Coste interno (mentor asignado)

Presupuesto total de formación: €20 000 (incluido en el presupuesto global de €450 000). Coste por desarrollador: ~€1 250/año, en línea con benchmark del sector financiero.

Compatibilidad con el editor WYSIWYG

El briefing es explícito: la solución **no puede romper la funcionalidad del editor** que usan los administradores para personalizar reportes regulatorios. La solución NO restringe lo que el administrador puede escribir; restringe lo que se renderiza al guardar:

1. El usuario escribe HTML libre en el editor (incluso `<script>` malicioso por error o por test).
2. Al hacer click en "Guardar", el contenido pasa por `DOMPurify.sanitize()` antes de persistirse.
3. El HTML peligroso se elimina; el formato visual (negrita, tablas, listas, enlaces seguros) se mantiene.
4. La preview muestra el HTML limpio aplicando `SafeHtmlPipe`.
5. La demo del prototipo en `/admin/templates` muestra los dos paneles lado a lado: HTML crudo escrito vs HTML sanitizado guardado, con un botón "Inject test payload" para que el evaluador lo verifique en vivo.

SECCIÓN 6 · Roadmap de Refactoring Incremental

El briefing es claro sobre el problema: **108 componentes con más de 800 líneas y 12 servicios "dios"** que concentran lógica de negocio, llamadas HTTP, transformación de datos y gestión de estado en un único archivo. **Una feature de 2 días lleva 6-8** porque nadie se atreve a tocar código que podría romper algo en otro módulo.

El plan de refactoring **no detiene el desarrollo de nuevas funcionalidades**. Los equipos siguen entregando features durante los 6 meses, y el refactor se aplica a los componentes que se tocan en cada Sprint, priorizando por impacto.

Patrones a aplicar (orden de prioridad)

Patrón	Problema que resuelve	Cómo se aplica
Container / Presentational	God Components que mezclan HTTP + estado + UI	Extraer presentational stateless con <code>input()</code> / <code>output()</code> . Container retiene servicios y lógica.
Repository pattern	Servicios "dios" con HTTP + transformación + estado	Separar acceso a datos (<code>*Service</code>) de telemetría (<code>*MetricsService</code>). Componente ya no llama directo a HTTP.
Inyección con <code>inject()</code>	Constructores con 8-12 parámetros	Reemplazar <code>constructor()</code> por <code>inject()</code> . Reduce boilerplate y mejora tree-shaking.
Angular Signals	BehaviorSubject + suscripciones + memory leaks	Migrar estado a <code>signal()</code> / <code>computed()</code> / <code>effect()</code> . Cancelación automática.
<code>ChangeDetectionStrategy.OnPush</code>	180+ ciclos de Change Detection	Aplicar <code>OnPush</code> universal. Solo recálculo cuando inputs cambian.
Control flow nuevo (<code>@if</code> / <code>@for</code> / <code>@switch</code>)	Templates con <code>*ngIf</code> / <code>*ngFor</code> legacy	Migración automática con <code>ng generate @angular/core:control-flow</code> .

Patrón	Problema que resuelve	Cómo se aplica
takeUntilDestroyed()	Memory leaks en suscripciones	Reemplazar Subject<void>() + takeUntil() por takeUntilDestroyed().
Standalone Components	NgModules acoplados	Migración con ng generate @angular/core:standalone.
Tipado estricto	any sin tipar, errores en runtime	strict: true + noImplicitReturns en tsconfig. Migración módulo a módulo.

Plan de descomposición de los 108 God Components

Fase	Acción	Output esperado
Sprint 1 (sem 3-4)	Inventario automático: script que detecta componentes > 800 LOC y servicios > 500 LOC. Cruce con frecuencia de modificación (git log) para priorizar los más volátiles.	Lista priorizada de 108 componentes + 12 servicios "dios" con métricas: LOC, frecuencia de cambio, número de commits últimos 90 días, dependencias.
Sprint 2 (sem 5-8)	Refactor de los 5 más usados/volátiles (regla 80/20: el 20 % más volátil concentra el 80 % de los bugs).	Container Pattern aplicado a los 5 críticos. ~30-40 componentes presentacionales nuevos. Cobertura local de tests > 70 % en estos archivos.
Sprint 3 (sem 9-14)	Refactor por equipo: cada uno aborda los God Components de su módulo durante el Sprint en que migra a MFE. Pair-programming con arquitecto senior.	~50 % de los 108 refactorizados (54 componentes). Patrón replicable documentado en wiki interna.
Sprint 4 (sem 15-18)	Resto de God Components + migración total a Signals. Servicios "dios" 100 % descompuestos.	~85 % refactorizados. Cobertura global > 30 %.
Sprint 5-6 (sem 19-26)	Long tail: componentes restantes + cero any. Migración a control flow nuevo automatizada con schematics.	100 % refactorizados. Cobertura > 50 %. Cero any en módulos productivos.

Cómo medir el progreso

- **Inventario automático en CI:** el script de detección corre en cada push a main y publica un dashboard con LOC por componente, evolución mensual y top 20 archivos más volátiles.
- **Análisis estático en CI** (a evaluar entre SonarQube, SonarLint local + ESLint reglas estrictas, o equivalente): bloquea PR que aumenten complejidad ciclomática, código duplicado o cobertura local en el archivo modificado por debajo de 70 %. Decisión final en Sprint 1.
- **Métricas semanales al CTO:** número de God Components restantes (objetivo: -15 %/Sprint), cobertura global de tests, ciclos de Change Detection medios.

- **Definition of Done por PR:** refactor con tests verdes, cobertura $\geq$ baseline, sin any nuevos, sin bypassSecurityTrust* sin justificación, sin markForCheck manual.

Mitigación de riesgos del refactor

- **Refactor sin tests es peligroso.** Cada refactor exige escribir tests previos sobre el comportamiento actual antes de tocar el código (Test-Driven Refactoring).
- **Feature flags por módulo refactorizado:** el código antiguo y el nuevo coexisten temporalmente. Si el refactor introduce regresión en producción, se desactiva la flag y se vuelve al antiguo en segundos.
- **Rollout progresivo:** primero a 10 % de usuarios (canary), luego 50 %, luego 100 %. Si cae métrica, rollback inmediato.

SECCIÓN 7 · Librería de Componentes Compartidos

La librería `@capitalflow/shared-ui` elimina las siete implementaciones de DataGrid, las cinco de Modal y los tres sistemas de notificaciones que existen hoy. Materializa el design system de Figma del equipo de diseño en componentes Angular reutilizables. Está **construida y verificada** en el prototipo.

Inventario verificado del prototipo

Categoría	Cantidad	Detalle
Componentes standalone	19	Button, Alert, Input, Modal, Tabs, Tab, Tooltip, Table, DataGrid, StatusBadge, Spinner, StatCard, MetricCard, InfoCard, Header, Footer, Icon, DonutChart, TrendChart
Pipes	2	IbanPipe (formato y ocultación parcial), SafeHtmlPipe (DOMPurify wrapper centralizado)
Directivas	2	ClickOutsideDirective, CapCellTemplateDirective
Services	1	DynamicCssService (inyección de reglas CSS en runtime, hardened con appendChild + textNode)
Tests automatizados	158	Cobertura de los componentes críticos (DataGrid, SafeHtmlPipe, validadores)
Stories de Storybook	19	Una por componente, con autodocs, ejemplos visuales y controles interactivos

Cómo se mantiene compatible con los diferentes setups de cada equipo

- **peerDependencies: $\geq 17.0.0$** : cubre los equipos en Angular 17, 18 y futura migración a 19+. El equipo Compliance (actualmente en Angular 16, sin LTS) migra a Angular 18 en Sprint 1-2 antes de integrar shared-ui — el orden es **migración primero, adopción después** para evitar tener que mantener un fork del design system con APIs reactivas antiguas. Las APIs que usa shared-ui (Signals, control flow `@if/@for`) son estables desde Angular 17.
- **Distribución como artefacto dist** (formato APF — Angular Package Format): ng-packagr genera FESM, type definitions y metadata. Los consumers no necesitan instalar las dependencias internas de la librería.
- **Importación única desde el entry point principal:** cero deep imports. Esto protege contra breaking changes en la estructura interna de la librería.

- **Standalone Components:** cada componente se importa individualmente. Tree-shaking efectivo: si un MFE solo usa Button, no carga el resto.
- **Pipeline CI específico:** la librería se construye una vez (build:shared-ui) y los MFEs consumen el dist como artifact, sin reinstalar deps. Tests cross-version validados antes de cada release.

Cómo evoluciona sin romper a los consumidores

Mecanismo	Garantía
Versionado SemVer estricto (1.x.y)	Las versiones 1.x son retrocompatibles. Breaking changes solo en mayor (2.0).
Changelog detallado por release	Cada release documenta cambios, depreciaciones y rutas de migración.
Anotación @deprecated en JSDoc	IDE muestra aviso al usar APIs depreciadas. Mínimo 2 versiones de margen antes de eliminar.
Tests de regresión visual con Storybook	Capturas comparadas en CI mediante herramienta de visual diff (Chromatic, Percy o similar; selección en Sprint 2): cualquier cambio visual no intencional bloquea el merge.
Storybook como contrato visual	Los equipos consultan Storybook como "documentación viva". Cualquier discrepancia con Figma es un bug.
Política de comunicación previa	Cambios mayores se anuncian 4 semanas antes en stand-up técnico cross-equipo.

Plan de adopción por los equipos

1. **Sprint 1:** shared-ui v1.0 con los 19 componentes core. Storybook accesible en localhost:6007. Documentación de migración por componente.
2. **Sprint 2-3:** cada equipo migra los componentes equivalentes en su módulo (DataGrid, Modal, formularios). Pair-programming con arquitecto senior.
3. **Sprint 4:** ESLint rule custom que prohíbe el uso de Angular Material o PrimeNG cuando existe equivalente en shared-ui. PR bloqueado automáticamente.
4. **Sprint 5-6:** completar la migración de los 8 equipos. Auditoría visual final con el equipo de diseño contra Figma.

Mapeo de migración: Material / PrimeNG / código propio → shared-ui

El briefing identifica tres setups distintos: **Material**, **PrimeNG**, y **código desde cero** (Auth). Tabla de mapeo de los componentes más usados:

Componente origen	shared-ui	Esfuerzo
mat-form-field + matInput / p-inputText	cap-input	Bajo — API compatible
mat-button / p-button	cap-button	Bajo — variantes mapeables
mat-table / p-table	cap-data-grid	Alto — 2-3 días por tabla compleja (CDK Virtual Scroll)
mat-dialog / p-dialog	cap-modal	Medio — focus trap, ARIA, backdrop

Componente origen	shared-ui	Esfuerzo
mat-tab-group / p-tabView	cap-tabs + cap-tab	Bajo
mat-snack-bar / p-toast	cap-alert + servicio	Medio — servicio en v1.2
mat-select / p-dropdown	cap-input variant select	Medio — Sprint 2 (CVA)
mat-tooltip / p-tooltip	cap-tooltip	Bajo
mat-icon	cap-icon	Bajo — set ~40 iconos en 2-3 días
Componentes Auth desde cero	cap-input + cap-button + cap-modal	Alto — 2 sprints, Container/Presentational

Estrategia de migración: cada equipo recibe un script que escanea su código en busca de imports de @angular/material o primeng y abre PRs automáticos para reemplazos triviales (botones, tooltips, iconos). Solo el DataGrid requiere intervención manual. Los 10 componentes listados cubren ~90 % de usos reales.

SECCIÓN 8 · Modernización del Despliegue

El estado actual es **artesanal**: deploy manual de 4-5 horas por SSH a tres máquinas virtuales, ejecutando scripts bash sin actualizar desde 2022, con **28 % de tasa de fallos** y rollbacks nocturnos de 2-3 horas. Los inversores preguntaron explícitamente: **"¿Cómo garantizan continuidad de servicio para clientes regulados?"**. La respuesta de esta sección es **zero-downtime para clientes regulados, garantizado por SLA**.

Pipeline GitLab CI/CD del prototipo

18 jobs en 4 stages, totalmente automatizado. El pipeline está **scoped por proyecto con `rules:changes`**: un cambio en mfe-payments dispara solo sus 4 jobs (los demás equipos no se bloquean); un cambio en shell o shared-ui dispara los 18 (son productos cross-cutting que validan todo); un push directo a main dispara los 18 siempre como safety net de regresión. Es exactamente el desbloqueo de equipos que motiva la migración a MFE: el deploy de Pagos ya no bloquea a los 7 equipos restantes.

Stage	Jobs	Función
build (5)	build:shared-ui, build:shell, build:mfe-payments, build:mfe-transactions, build:mfe-analytics	Compilación. shared-ui produce artifact dist consumido por los demás. Builds en paralelo cuando sea posible.
test (5)	test:shared-ui, test:shell, test:mfe-payments, test:mfe-transactions, test:mfe-analytics	354 tests automatizados (Karma + Jasmine para Angular, Jest para React). Bloqueo si alguno falla.
docker (4)	docker:shell, docker:mfe-* (×3)	Build de imágenes Docker multistage (Node 20 alpine + nginx stable alpine). Las 5 imágenes compilan end-to-end localmente (shared-ui se construye dentro del contenedor antes del build del MFE Angular), tamaño final 62-72 MB por imagen.

Stage	Jobs	Función
deploy (4)	deploy:shell, deploy:mfe-payments, deploy:mfe-transactions, deploy:mfe-analytics	Cada surface se despliega independientemente con <code>when: manual</code> (aprobación requerida). Modelo blue/green y canary descritos abajo se aplican dentro de cada job vía <code>kubect!/ArgoCD</code> en F4.

Estrategia de zero-downtime para clientes regulados

La continuidad de servicio se garantiza mediante una combinación de cinco mecanismos:

Mecanismo	Garantía técnica	SLA al cliente regulado
Blue/green deployment	Dos entornos productivos idénticos (blue activo, green standby). El deploy actualiza green; tras smoke tests, el load balancer (nginx upstream) conmuta tráfico a green. Blue queda como rollback caliente.	Cero pérdida de sesión. Conmutación < 30 s.
Health checks por servicio	Cada MFE expone <code>/health</code> que verifica conexión a backend, idioma cargado, dependencias críticas. El load balancer solo enruta a instancias healthy.	Si una instancia falla, el tráfico se re-enruta automáticamente.
Rollback automatizado	Si los smoke tests post-deploy fallan o el error rate supera el umbral en los primeros 10 min, ArgoCD revierte automáticamente.	Rollback < 2 min vs 2-3 h manuales actuales.
Canary releases	El nuevo deploy se libera primero al 10 % de usuarios. Si métricas (error rate, latencia, FCP) están dentro de targets durante 30 min, se sube al 50 % y luego al 100 %.	Cliente regulado nunca ve cambios sin validación previa.
Ventanas de mantenimiento opcionales	Para clientes regulados con SLA estricta, deploys programados fuera de horario laboral con notificación previa de 48 h.	Comunicación proactiva, contractualizada.

Cumplimiento de auditoría

- **Logs centralizados** con retención de 12 meses (requisito SOC 2). Cada deploy genera trail completo: quién, cuándo, qué versión, resultado.
- **Aprobaciones manuales para producción** (`when: manual` en GitLab) firmadas por el responsable del equipo. Nadie despliega solo.
- **Imágenes firmadas con Cosign**: cada imagen Docker se firma criptográficamente. El cluster solo acepta imágenes con firma válida del registry corporativo.
- **Health gating post-deploy**: el deploy se considera exitoso solo cuando los health checks pasan durante 5 min consecutivos. Si no, rollback automático.

Roadmap del pipeline productivo (6 meses)

Sprint	Mejora	Beneficio
1 (sem 1-4)	Pipeline básico operativo + npm audit + gitleaks en cada PR	Bloqueo automático de credenciales filtradas y dependencias vulnerables
2 (sem 5-8)	Trivy y Semgrep (SAST) integrados. Registry corporativo + docker push reales. Cosign para firma.	Cumplimiento SOC 2 Type II
3 (sem 9-14)	npm ci --ignore-scripts para builds reproducibles. Lighthouse CI con performance budgets que bloquean PR si regresan.	Defensa contra postinstall malicioso. Rendimiento garantizado.
4 (sem 15-18)	Blue/green operativo con kubectl o ArgoCD. Canary 10/50/100 con auto-rollback.	Zero-downtime real, no solo prometido.
5 (sem 19-22)	Observabilidad completa: Prometheus + Grafana + sistema de alertas oncall. Métricas de SLA en dashboard ejecutivo.	Visibilidad para el Board.
6 (sem 23-26)	Chaos engineering en staging: caída controlada de un MFE para validar resiliencia.	Confianza en el sistema.

SECCIÓN 9 · Plan de Ejecución (6 meses)

El plan se divide en **6 fases secuenciales** alineadas con los hitos de negocio (due-diligence, SOC 2, renovación de cliente Francia). Las fases se solapan parcialmente para maximizar paralelización del equipo.

Fases, entregables y criterios de éxito

Fase	Semanas	Entregables principales	Criterio de éxito
F1. Fundamentos + Compliance	1-4	Shell + 3 MFEs + shared-ui v1.0 + CI/CD básico + 7 security headers + matriz de vulnerabilidades demostrable + migración Compliance A16→A18 (Sprint 1-2)	Pipeline verde, headers activos, demos vivas, cobertura > 10 %. Compliance ya en A18 e integrado en shared-ui.
F2. Rendimiento + Due-diligence	5-8	Virtual scroll + Web Worker + OnPush + lazy loading + legacy modal (IE11) + métricas Lighthouse + SOC 2 Type II	FCP < 4 s, tabla 80 K < 1 s, bundle < 800 kB. Due-diligence superado. SOC 2 certificado con Compliance en versión LTS.
F3. Migración MFEs prioritarios	9-14	Treasury y resto de equipos extraídos a MFEs independientes	Deploys independientes operativos para 5/8 equipos. Zero-downtime verificado.

Fase	Semanas	Entregables principales	Criterio de éxito
F4. Refactoring + observabilidad	15-18	Descomposición de 50-60 % de los God Components. Migración masiva a Signals. Grafana + alertas.	Change Detection < 40 ciclos. Cobertura > 30 %. Tiempo de feature: 4 días (vs 6-8).
F5. Consolidación	19-22	Onboarding, Admin y Auth migrados a MFE. Portal legacy IE11 entregado.	Los 8 equipos con pipeline propio. 6 % usuarios IE11 con servicio garantizado.
F6. Optimización final	23-26	Bundle < 300 kB, tests > 50 %, blue/green operativo, chaos engineering, presentación al Board.	Todos los KPIs cumplidos para Serie B.

Equipo y dedicación

Rol	Personas	Dedicación	Responsabilidades
Arquitecto consultor	1	100 %	Diseño arquitectónico, decisiones técnicas, formación, code review semanal cross-equipo
Arquitectos senior internos	2	50 %	Implementación de MFEs core, mentoring, revisión de PRs, pair-programming en migración
Desarrolladores (8 equipos)	14	20 %	Migración incremental de sus módulos, refactor por sprint, pair con arquitecto cuando toque su MFE

Presupuesto

Concepto	Importe	Detalle
Consultor externo (Angular Expert)	€180 000	6 meses × 100 % dedicación
2 arquitectos senior (50 %)	€120 000	Coste interno asignado a este proyecto
14 desarrolladores (20 %)	€90 000	Coste interno asignado
Formación del equipo	€20 000	OWASP, formación Secure Coding, materiales (detalle en Sección 5)
Herramientas y licencias	€18 000	Análisis estático CI, regresión visual, observabilidad, alertas oncall
Infraestructura cloud (staging + prod)	€15 000	Sobrecoste durante migración (paralelización blue/green)
Buffer de contingencia (≈ 1,6 %)	€7 000	Reservado para imprevistos

Concepto	Importe	Detalle
TOTAL	€450 000	Coherente con el constraint del briefing

Dependencias entre fases

- **F2 depende de F1:** no se puede optimizar lo que no está aislado en MFEs.
- **F3 depende de F2:** los MFEs migrados deben heredar las optimizaciones de rendimiento del Shell.
- **F4 puede correr en paralelo a F3** desde la semana 12 (refactor por equipo, no global).
- **F5 y F6 secuenciales** después de F3 para no saturar al equipo.

Hitos críticos para el negocio

Semana	Hito	Stakeholder
Semana 6	Certificación SOC 2 Type II completada ($\leq$ 45 días desde firma)	Compliance, inversores
Semana 8	Due-diligence Serie B presentado con prototipo, métricas y plan	Inversores
Semana 12	Ciente Francia con mejora demostrable de rendimiento (renovación contrato €420 K)	CEO, comercial
Semana 14	4/8 equipos con pipeline independiente	CTO
Semana 22	8/8 equipos migrados a MFEs	CTO, equipos
Semana 26	Métricas finales para presentación final al Board	Board, inversores

SECCIÓN 10 · Análisis de Riesgos

Identificación de los riesgos principales con probabilidad, impacto y plan de mitigación. Los riesgos están priorizados por **probabilidad × impacto** y revisados quincenalmente con el CTO.

Riesgo	Prob.	Impacto	Mitigación
Resistencia del equipo Analytics a integrar Web Components	Media	Alto	Prototipo funcional ya operativo. Cero cambios al código React. Demo y formación en semana 1. Pair con uno de sus desarrolladores.
Regresiones durante migración a MFE	Alta	Medio	Feature flags por MFE + rollback automatizado + tests E2E Playwright en cada PR + canary 10/50/100.
Retraso en certificación SOC 2 Type II	Baja	Crítico	9 de 11 vulnerabilidades implementadas o demostrables en el prototipo. Las 2 restantes (cookies HttpOnly, CSP nonces) en Sprint 1 con responsables asignados.

Riesgo	Prob.	Impacto	Mitigación
Incompatibilidad Angular 16 con shared-ui	Media	Alto	peerDependencies >= 17. Para Compliance se evalúa fork con >= 16 en Sprint 4. APIs usadas (signals, control flow) disponibles desde A16 reciente.
Pérdida de rendimiento durante transición	Media	Medio	Métricas Lighthouse en cada PR. Performance budgets en angular.json. Alertas si FCP regresa > 10 % vs baseline.
Fallo en deploy a producción	Media	Crítico	Blue/green + health checks + rollback < 2 min. Aprobación manual obligatoria. Smoke tests post-deploy.
Adopción lenta de shared-ui	Media	Medio	Storybook como referencia visual obligatoria. ESLint rules que impiden Material/PrimeNG cuando existe equivalente. Demos en stand-up semanal.
Bus factor: un único arquitecto consultor	Baja	Crítico	2 arquitectos senior internos al 50 %, pair-coding desde semana 1. Documentación cross-equipo. Plan de transferencia de conocimiento desde mes 4.
Desviación presupuestaria	Baja	Alto	Buffer del 1,6 % en presupuesto. Revisión quincenal con CTO. Plan de contingencia: priorizar Sprints críticos (F1+F2+F3) si hay recortes.
Resistencia a abandonar Material/PrimeNG	Media	Medio	shared-ui cubre 90 % de casos de uso reales auditados. Storybook visual demuestra paridad. Migración incremental, no big-bang.

Plan de contingencia para retrasos

1. **Retraso ≤ 1 semana:** revisión de dependencias entre fases. Negociación con CTO para mover features no-críticas al Sprint siguiente.
2. **Retraso > 1 semana:** incremento de dedicación de arquitectos senior (50 % → 75 %) durante el sprint problemático. Comunicación temprana al Board.
3. **Riesgo de superar el due-diligence (sem 8):** priorización absoluta de F1+F2 (fundamentos + rendimiento + SOC 2). El resto de fases puede ajustarse.
4. **Comunicación de riesgos:** dashboard de riesgos compartido con CTO. Riesgos en estado "Activo" se reportan al Board en informe mensual.

SECCIÓN 11 · Métricas de Éxito

Las métricas se dividen en **KPIs técnicos** (verificables por el evaluador y por el equipo de inversores) y **KPIs de negocio** (impacto en cliente y operación). Todos tienen baseline actual, target intermedio (semana 8) y target final (semana 26).

KPIs técnicos

KPI	Baseline	Sem 8	Sem 26	Cómo medir
Bundle inicial Shell	2,8 MB	< 800 kB	< 300 kB	webpack-bundle-analyzer + budgets
First Contentful Paint (4G)	9,2 s	< 4 s	< 2,5 s	Lighthouse CI semanal
Time to Interactive	14 s	< 6 s	< 3 s	Lighthouse CI
Tabla 80 K registros	18 s congela	< 1 s	< 300 ms	Chrome DevTools Performance
Export Excel 120 K filas	Bloquea UI	Worker	< 5 s sin bloqueo	Custom metric exportPhase
Tiempo de deploy productivo	4-5 h	30 min	< 5 min/MFE	GitLab CI duration
Tasa de fallo en deploy	28 %	< 10 %	< 1 %	GitLab statistics
Cobertura de tests	3 %	> 30 %	> 50 %	Karma coverage (Istanbul)
Vulnerabilidades alta/crítica	11	≤ 2	0	npm audit + auditoría externa anual
Ciclos Change Detection	180+	< 40	< 20	Angular DevTools Profiler
Tiempo rebuild completo (dev)	14 min	< 3 min	< 1 min	GitLab CI + medición local

KPIs de negocio

KPI	Baseline	Target	Plazo
Tiempo de entrega de features	6-8 días	2 días	Fase 4
Incidentes cross-equipos / mes	3-4	0	Fase 2
NPS de clientes enterprise	32	> 50	Fase 4
Certificación SOC 2 Type II	Bloqueada	Completada	≤ Semana 6
Renovación contrato Francia (€420 K/año)	En riesgo	Asegurada	Fase 2
Coste mantenimiento mensual (h)	~600 h	~350 h	Fase 5
Onboarding nuevo dev (días → primer PR)	~8	~2	Fase 4
Tasa de fallos en producción	Sin medir	< 0,5 % requests	Fase 5 (tras observabilidad)

Cadencia de reporte al Board

- **Semanal:** dashboard automatizado de métricas técnicas (Lighthouse, GitLab CI, npm audit, cobertura). URL accesible al CTO 24/7.

- **Mensual:** informe ejecutivo de progreso vs plan, KPIs de negocio actualizados, riesgos activos y notas técnicas resumidas.
- **Por fase:** presentación al Board con demo en vivo del prototipo y métricas reales contra targets de la fase.
- **Mes 2 (semana 8):** due-diligence Serie B con prototipo + plan + métricas baseline → target.
- **Mes 6 (semana 26):** presentación final al Board e inversores con métricas alcanzadas y compromiso para próximos 12 meses.

ANEXO A · Respuesta detallada a la pregunta del inversor

"Si el equipo de Analytics quiere seguir en React para siempre, ¿cómo consigues que los usuarios vivan una experiencia unificada y el equipo de diseño mantenga consistencia visual sin forzar una migración que ellos rechazarán?"

Respuesta corta

Custom Elements API + Shadow DOM + CSS Custom Properties como design tokens cross-framework. Estándar W3C, cero acoplamiento, demostrado y funcionando en el prototipo en /analytics.

Cómo funciona técnicamente

1. **React se compila a un Web Component:** el código React se empaqueta en una clase que extiende `HTMLElement` y se registra con `customElements.define("mfe-analytics", ...)`. El equipo Analytics no toca su código de aplicación; sólo añade un wrapper de ~40 líneas que React monta dentro del Shadow DOM.
2. **Shadow DOM `mode: open`** aísla los estilos React del Shell y viceversa. Cero conflicto de CSS entre frameworks. El equipo Analytics sigue usando la librería de estilos que prefiera (Styled Components, CSS Modules, etc.) sin riesgo.
3. **Angular consume el Web Component como HTML nativo:** `<mfe-analytics></mfe-analytics>` con `CUSTOM_ELEMENTS_SCHEMA`. Sin imports, sin migración, sin acoplamiento.
4. **CSS Custom Properties cruzan el Shadow DOM con fallbacks:** `var(--cap-primary, #2a85c4)`. El Shell define los tokens en `<html>`, atraviesan el Shadow DOM y React los aplica. Si el Shell no los define, React usa el fallback.
5. **El idioma se sincroniza con `MutationObserver`:** el Shell escribe `document.documentElement.lang = "es"`, React lo detecta con un Observer y cambia su locale automáticamente. Sin librería compartida ni event bus custom.

Garantías para el equipo de Analytics

- **Cero modificación del código de aplicación:** solo añadir el wrapper de Web Component (~40 LOC, una vez).
- **Stack React intacto:** pueden seguir usando React Query, Redux, su librería de UI, su pipeline de tests Jest. El prototipo demuestra esto con 26 tests Jest del MFE Analytics pasando junto a los 328 Karma de los proyectos Angular.
- **Pipeline de despliegue independiente:** CI/CD propio para Analytics. Releases sin coordinación con los equipos Angular.
- **Cero riesgo de breaking change desde el Shell:** el contrato es HTML estándar. Si el Shell cambia, no afecta al MFE.

Garantías para el equipo de diseño y los usuarios

- **Consistencia visual:** los tokens (`--cap-primary`, `--cap-bg`, `--cap-text`, etc.) se definen en un único lugar (`shared-ui/src/styles/tokens.scss`). React los respeta automáticamente.
- **Tema claro/oscuro unificado:** cambiar el tema en el Shell cambia automáticamente el aspecto del MFE React, sin tocar su código.
- **Idioma sincronizado:** cambiar idioma en el Shell propaga al MFE en milisegundos.
- **Componentes compartidos disponibles:** los componentes complejos del design system (DataGrid, Modal) se exponen también como Web Components, y React los puede consumir si quiere reaprovechar lógica.

Resultado verificado en el prototipo

La ruta `/analytics` del Shell carga el MFE React. El usuario ve un dashboard con la misma identidad visual que el resto de la plataforma. Toggle de tema y cambio de idioma funcionan transparentemente. El equipo Analytics no ha tocado su código de aplicación: solo añadió un wrapper de Web Component de ~40 líneas.

El experimento funciona. La pregunta del inversor tiene respuesta técnica concreta y verificable.

ANEXO B · Mapa de archivos del prototipo

Para que el evaluador pueda comprobar cada decisión de esta propuesta en el código real entregado.

Equivalencia entre nomenclatura del briefing y carpetas del repositorio

Los nombres de carpeta del repositorio reflejan el scaffolding inicial; la nomenclatura de equipos del briefing es la canónica y la que se aplicará en el rename programado para el primer Sprint productivo.

Equipo (briefing)	Carpeta en el repositorio	Estado
Team Payments	mfe-payments/	Coincide ✓
Team Treasury	mfe-transactions/	Rename a mfe-treasury planificado en Sprint 1
Team Analytics	mfe-analytics-react/	Sufijo -react del scaffolding; rename a mfe-analytics planificado en Sprint 1

Los nombres de jobs del pipeline GitLab (`build:mfe-transactions`, `test:mfe-transactions`, `build:mfe-analytics`, `test:mfe-analytics`) se actualizan en el mismo commit del rename. Hasta entonces, la nomenclatura del repositorio se mantiene estable para no introducir regresiones en pipelines verdes.

Mapa de archivos por concepto

Concepto	Archivo
Shell — routing y lazy loading	shell/src/app/app.routes.ts
Shell — wrapper de MFE + loader Module Federation	shell/src/app/core/
Shell — reverse proxy <code>/remotes/*</code> (dev y prod)	shell/proxy.conf.json y shell/nginx.conf
Shell — security headers nginx	shell/nginx.conf
Shell — matriz de las 11 vulnerabilidades	shell/src/app/admin/admin-landing/admin-landing.constants.ts
Demo XSS WYSIWYG (vuln #5)	shell/src/app/admin/wysiwyg-editor/
Demo iframe URL allowlist (vuln #2)	shell/src/app/admin/pdf-viewer/
Demo XSS reflejado (vuln #4)	shell/src/app/search-demo/
Modal legacy (IE11)	shell/public/cap-modal-legacy.{js,css}
mfe-payments — bootstrap como custom element	mfe-payments/src/bootstrap.ts

Concepto	Archivo
mfe-treasury (carpeta mfe-transactions) — Container + Virtual Scroll	mfe-transactions/src/app/
mfe-treasury — Web Worker XLSX	mfe-transactions/src/app/workers/export.worker.ts
mfe-analytics (carpeta mfe-analytics-react) — custom element	mfe-analytics-react/src/web-component.tsx
shared-ui — DataGrid con CDK Virtual Scroll	shared-ui/src/lib/cap-data-grid/
shared-ui — SafeHtmlPipe (DOMPurify)	shared-ui/src/lib/safe-html/safe-html.pipe.ts
shared-ui — design tokens claro/oscurio	shared-ui/src/styles/tokens.scss
shared-ui — Storybook config	shared-ui/.storybook/
Pipeline GitLab CI/CD	.gitlab-ci.yml
Docker Compose con healthchecks	docker-compose.yml
Setup automático Windows / Linux-Mac	setup.ps1 y setup.sh
Tests E2E Playwright	e2e/specs/

Cierre

Esta propuesta articula un plan **concreto, ejecutable y validado** para modernizar CapitalFlow en 6 meses con €450 000 y el equipo disponible. La arquitectura propuesta está construida en un prototipo verificable, las métricas tienen baseline real y target medible, y el plan de ejecución respeta los hitos de negocio críticos: certificación SOC 2 en semana 6, due-diligence Serie B en semana 8 y plataforma productiva final en semana 26.

Los cuatro problemas centrales del briefing tienen respuesta técnica documentada: la fragilidad arquitectónica se resuelve con Module Federation, el rendimiento con CDK Virtual Scroll + Web Workers + lazy loading, las 11 vulnerabilidades con mitigaciones implementadas y plan de formación, y la integración del equipo React con Custom Elements API sin forzar migración.

La modernización de CapitalFlow no requiere un big-bang. Requiere disciplina, una arquitectura desacoplada y una hoja de ruta. Esta propuesta es esa hoja de ruta.

Francisco Liébanas — Angular Expert Candidate, Nter — Mayo 2026